

1

NOCK & THE ART OF
SOLID-STATE COMPUTING
An Enquiry into the Virtual

2

3

4

N. E. Davis
~lagrev-nocfep

5

6 Contents

7	1 The Roots of Computing in the Ancient	
8	World	1
9	1.1 Tabulation	3
10	1.2 Solid-state indices	3
11	2 Combinators for Fun and Profit	5
12	2.1 Combinators in Computing	6
13	2.1.1 The SKI calculus	6
14	2.2 Nock Opcodes as Combinator	7
15	2.3 Idioms & Design Patterns	7
16	2.4 Virtualization	7
17	3 Interpreter Runtime	9
18	Hoon Proper	11

Chapter 1

The Roots of Computing in the Ancient World

Abstract

Cuneiform and early computational concepts are explored in this chapter.

Machines are pure products of human intellect, constructed for man's own purposes. They are nowhere to be found in the world of nature (as products of nature); yet the workings of the laws of nature find their purest expression in machines, purer than in any of the products of nature itself. The laws of nature work directly in machines, with an immediacy not to be found in the products of nature.

37 (Keiji Nishitani, *Religion and Nothingness*,
38 1982, pp. 129–130)

39 There are many worthy raconteurs of the history
40 of computing, such as **Hofstadter1980 (Hofstadter1980)**
41 and **Penrose2004 (Penrose2004)**. What distinguishes
42 my approach to the history of computing from other
43 tellings? Computing has deep connexions to alchemy
44 and true language, which I hope to demonstrate with-
45 out assuming any Hermetic background on the reader's
46 part. Much like technology, computing is not merely
47 a tool but a stance towards the world. However, it
48 does not entail technologism's supercession; com-
49 puting is always both a lens and a kaleidoscope, through
50 which many layers of reality can be seen simultane-
51 ously. It is a way of seeing the world as a system
52 of pure statements and acting to make those state-
53 ments true. It is both ontology and epistemology.

54 First, therefore, I must establish that computa-
55 tion is the apex of a long human search for a lan-
56 guage of truth. The origin of symbol itself in the
57 σύμβολον, a token broken in half to infallibly mark
58 identity, is one font. Later "symbol" was the profes-
59 sion of faith, both an attempt to formulate truth and
60 a sign of shared claims. The primary source, how-
61 ever, is language itself, Hofstadter's "strange loop"
62 of self-reference. The first possible Nock formula is
63 a crash, the void. The second is self-reference, awak-
64 ening. Peirce would be proud of the progression. We
65 have tricked ourselves into thinking that computing
66 is a late development, a fruitful elaboration of the
67 end of mathematics. In fact, it is the philosopher's
68 stone, and it underlies every process in the world. It

69 has turned lead into gold.

70 The historical account which follows aims through-
71 out at the goal of understanding what Nock is and
72 why it matters as a protocol. The reader will under-
73 stand what mankind has been looking for, when we
74 have caught various pieces of the puzzle, how these
75 things came together in the mid-20th century, and
76 how they have been refined into a pure machine.

77 Outline: - Cuneiform, tabulation, calculation, and
78 algorithm - Solid-state indices: abaci, tokens, etc.
79 Mechanism as embodied equation. - Pure language
80 and true statements (Llull, Leibniz, Wilkins) - Gen-
81 eralization (Jacquard, Babbage) - The universal ma-
82 chine (Turing, Gödel, Zuse, von Neumann) - Pure
83 statements as Pages from the Book (Curry, Church)

84 1.1 Tabulation

85 1.2 Solid-state indices

86 The problem is that any calculation lives in some-
87 one's head or someone's claim. This is a summary,
88 but not the proof or solution.

89 Chapter 2

90 Combinators for Fun 91 and Profit

92 Abstract

93 The first section of this chapter will introduce the
94 basic SKI and BCKW combinator calculi and examine
95 how they can represent each other. The second sec-
96 tion will introduce Nock’s opcodes as combinators,
97 and show how they can be used to express various
98 combinator idioms. The third section will show how
99 to build practical affordances for common program-
100 ming tasks, such as conditionals, recursion, subpro-
101 cedure invocation, and data structures. The final sec-
102 tion will discuss the relationship between combina-
103 tors and virtualization, and how Nock can be used to
104 build a virtual machine even in a crash-only context.

105 2.1 Combinators in Computing

106 The pioneers of computing elaborated three comple-
107 mentary visions:

- 108 1. The Turing machine, a simple abstract machine
109 that can compute anything computable.
- 110 2. The lambda calculus, a pure functional language
111 based on substitution and abstraction.
- 112 3. Combinatory logic, a variable-free functional
113 language based on combinators.

114 Logicians early demonstrated the equivalence of
115 these approaches, but particular formulations are still
116 preferred for particular tasks. The Turing machine is
117 the most intuitive model of computation, and is of-
118 ten used to prove computability results. The lambda
119 calculus is the basis of many functional program-
120 ming languages, such as Lisp, Scheme, and Haskell.
121 Combinatory logic is less widely used in practice,
122 but has a long history in mathematical logic and type
123 theory. It is also the basis of Nock.

124 One of the most fundamental objectives of com-
125 binatory logic is to eliminate variables.

126 2.1.1 The SKI Calculus

127 Computable functions can be expressed using mini-
128 mal sets of combinators. Many logicians today pre-
129 fer to work with Moses Schönfinkel and Haskell Curry's
130 SKI combinator calculus, which has three combina-
131 tors:

132 I

133 K

134 S

135 B

136 C

137 W

138 U

139 Y

140 $Y = S (K (S I I)) (S (S (K S) K) (K (S I I)))$

141 Combinator calculi are generally considered too
142 minimalist for practical programming; they shine,
143 however, in mathematical proofs. This means that
144 we can use them to define a standard of behavior
145 for an algorithm, but implement the logic in a com-
146 pliant form using whatever computational tools we
147 like. Thus it is better to think of Nock not as a bare
148 programming language, but as a standard of compu-
149 tation which can *in extremis* be used as the compu-
150 tation itself.

151 2.2 Nock Opcodes as Combinator

152 2.3 I Identity = Opcode 0

153 Nock's binary tree addressing is fully general and
154 yields the identity combinator I as a special case.
155 (The cost, of course, is dragging in some machinery
156 for basic arithmetic, as we will see.)

157 2.4 K Constant = Opcode 1

158 2.5 S Substitution = Opcode 2

159 2.6 The Axiomatic Operators

160 Because of Nock's cellular nature as a binary tree,
161 we need the ability to access and manipulate nouns.
162 opcode 3, 4, 5, 10

163 2.7 Idioms & Design Patterns

164 Several of the combinators we examined above are
165 not present in Nock simpliciter, but we can build
166 them out of the other pieces. There are also common
167 idioms which are baked in as "compound opcodes"
168 (such as conditional branching, opcode 6).

169 Recursion we get for free with opcode 0.
170 μ -recursive functions

171 2.8 Virtualization

172 Nock is a crash-only pure language, which means at
173 the base level it can only succeed and yield a new
174 noun: no side effects, no exceptions, no state. Like
175 a castaway on a desert island, we must build every-
176 thing we need from raw materials at hand.

177 Nock has one trick built in and a couple of design
178 patterns to help us get there:

- 179 1. Opcode 11 lets us send a hint to the runtime,
180 which can be used to trigger side effects. The
181 Nock expression formally doesn't know any-
182 thing about these, but we can use them to print
183 results, read input, and so forth.
- 184 2. The runtime can maintain state outside of Nock
185 and supply it as a subject to subsequent Nock
186 expressions. We will do a lot more with this
187 later, but we want to highlight it as a key af-
188 fordance now.
- 189 3. The virtualization pattern means that we can
190 run Nock in Nock.

191 Opcode 11 can encode anything we want: the re-
192 sult of a hint is used by the runtime, and so anything
193 the runtime is willing to work with can be produced.

194 The magic formula at the beating heart of a prac-
195 tical Nock operating system is simply:

```
196       ★([state -.events] [9 2 10 [6 0 3] 0 2])  
197  
198
```

199 That is, apply the Nock formula against a subject pair
200 consisting of the current state and the head of the list
201 of events to process.

202 Chapter 3

203 Interpreter Runtime

204 Abstract

205 Now that we have established a firm foundation
206 for idiomatic programming in Nock, we can build an
207 interpreter using a higher-level language to execute
208 Nock formulas. At this point, we will build the sim-
209 plest possible Nock interpreter, a tree-walking inter-
210 preter. This requires a cursor and some basic mem-
211 ory management but not much more. We will im-
212 plement some minimalist static hints to permit side
213 effects like output.

²¹⁴ Hoon Proper